College of Business Education

Data Structure and Algorithm

BIT 2 Semester 1

Introduction

What is a data structure?

A data structure is a way of organizing data so that it can be efficiently utilized. Almost all software systems and programs employ many basic and complex data structures. Therefore, we must be knowledgeable in data structures.

What is an algorithm?

Refers to a sequence of actions taken to complete a particular task or to get the intended result. Algorithms are clearly defined procedures that help computers solve problems. It's like the building blocks that make it possible for computing devices to work well and make decisions. Each line of code the program writes is an algorithm that tells it what to do.

Relationship between data structure and algorithm

Algorithms have been used to manipulate data structures across different programming languages. On the other hand, data structure helps improve an algorithm's performance. Therefore, algorithms and data structures depend on each other. For example, the algorithm can perform the search, insert, update, delete, and sort operations on the data in the data structure

Why are data structures and algorithms essential?

Data structures and algorithms help us store and process data efficiently, considering time and memory usage, which improves the performance of computer programs and computer systems.

Data types

In computer science and computer programming, a data type (or simply type) is a set of possible values and the operations that can be performed on them. A data type tells the compiler or interpreter how the programmer intends to use the data. The data type defines two important things.

1) Defines a certain domain of values (It determines which values can be stored)
2) Define the operation allowed on those values

Example:

For int type

- It takes only an integer value
- Operations: Addition, Subtraction, multiplication, division, bitwise operations etc.

For float type

- It takes only a floating value
- Operations: Addition, Subtraction, multiplication (bitwise operations and % are not allowed)

There are two types of data:

- **Primitive data types:** A primitive data type specifies the size and type of variable values. These are data types built into a programming language and can be used to derive other data types (non-primitive data types). Unlike non-primitive types, primitive data types store only one value. Examples of primitive data types include int, float, and double.
- **Non-primitive data types (Abstract data types).** These are user-defined data types used to define the values of non-primitive data types. The user defines the values and operations manually. On the other hand, the system can also determine the operations using the built-in function. Abstract data types can store more than one value simultaneously. Examples of non-primitive data types are arrays, structures, and unions.

Since data type operations involve storing and manipulating data in memory, they are also called data structures. There are two types of data structures:

- Primitive data structure
- Non-primitive data structure

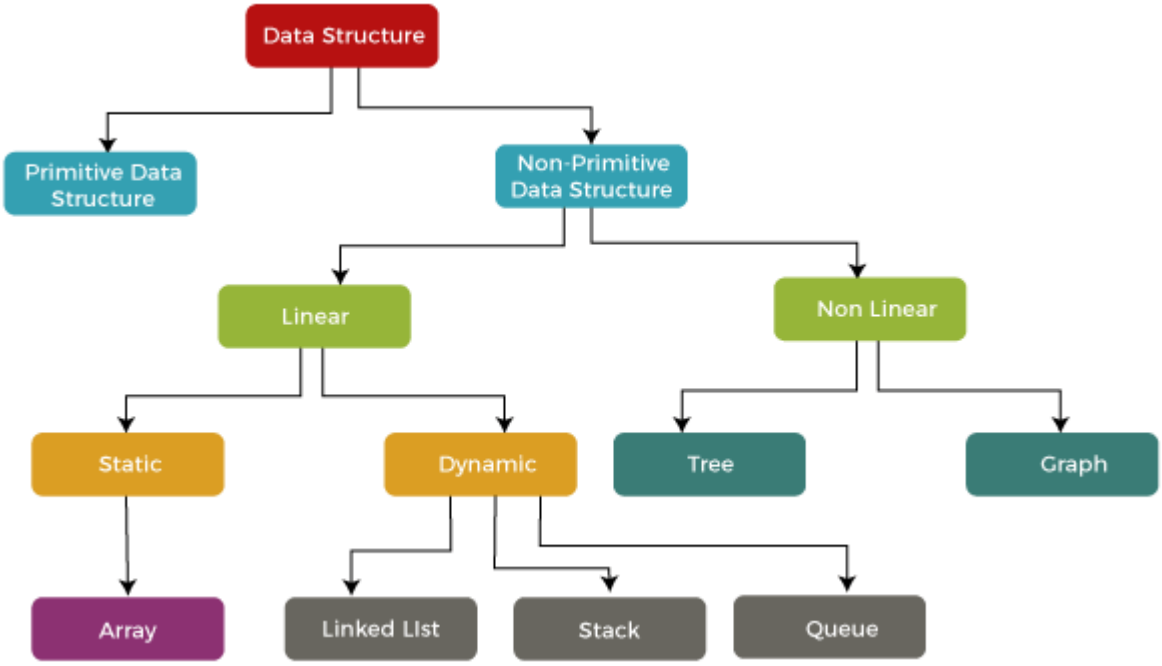


Figure 1: Classification of data structure

Primitive Data structure

The primitive data structures are primitive data types. The primitive data structure is a fundamental data structure that stores only one data simultaneously. The int, char, float, double, and pointer are primitive data structures that can hold a single value.

Non-Primitive Data structure

The non-primitive data structure is a type of data structure which is user-defined that stores the data of different types in a single entity.

- **Linear data structure**

  A linear data structure is one in which data items are organized sequentially or linearly, with each component connected to the elements before and after it. Examples of the linear data structure are array, stack, queue and linked list

- **Static data structure**

  The size of the structure in a static data structure is fixed. It is possible to change the data structure's content without impacting the amount of memory space assigned to store the data. An example of a static data structure is an array

- **Dynamic data structure**

  The size of a dynamic data structure is not fixed and may be changed while operations are being performed on it. Dynamic data structures are intended to allow for data structure changes during runtime. Example of dynamic data structure is Que, stack and linked list

- **Non-linear data structure**

  Non-linear data structures are those in which data components are not arranged sequentially or linearly. Therefore, we cannot explore all of the items of a non-linear data structure in a single run. Examples of non-linear data structures are trees and graphs

Major Operations which can be performed in the data structure

The major or common operations that can be performed on the data structures are:

- **Searching:** We can search for any element in a data structure
- **Sorting:** We can sort the elements of a data structure either in an ascending or descending order
- **Insertion:** We can also insert a new element in a data structure.
- **Updation:** We can also update the element, i.e., we can replace the element with another element
- **Deletion:** We can also perform the delete operation to remove the element from the data structure

Functions/Methods

A Java method is a collection of statements grouped to perform a certain operation in a Java program. For example, a section of a bank information system dealing with customers' deposits may contain several lines of code that communicate with the database, which transfers and stores the desired amount in the customer's account. All program statements that perform deposits can be grouped into a function.

The syntax for creating a function

access_specifier static return_type method name (list of arguments)

```
{
    //Function Body
}
```

Example of function

```
public static float average (float marks)
{
   //Codes for calculating the average go here
}
```

Definition of terms from the function above

public: An access specifier which indicates that the function can be accessed outside the class

static: Indicates that the **function** can be called without creating an **object.** Alternatively, we can say that no **object** owns this function.

float: It is the **return** type which indicates that our **function** will return average marks as a float value. Otherwise, it should be declared void if the function returns nothing.

average: It is a class name

marks: parameter variable (This part is optional)

Question: Write a Java program that implements a function concept that calculates the average of three input Marks, such as Mathematics, Geography, and English, and outputs the results on the screen.

```java
package BICT_DAY;
import java.util.Scanner;
public class Main {
    public static void evarage()
    {
        float math;
        float geog;
        float engl;
        Scanner input=new Scanner(System.in);
        System.out.println("Enter Mathematics Marks");
        math=input.nextFloat();
        System.out.println("Enter Geograph Marks");
        geog=input.nextFloat();
        System.out.println("Enter English Marks");
        engl=input.nextFloat();
        float average=(math+geog+engl)/3;
        System.out.println("Average Mraks is "+average);
    }
    public static void main(String[] args) {
        // TODO code application logic here
        evarage();
    }
}
```

Recursion

Recursion in java is a process in which a method calls itself continuously. A method in java that calls itself is called recursive method.

It makes the code compact but complex to understand.

Syntax

returntype methodname()

{

//code to be executed

methodname();//calling same method

}

Example: A program to display 'hello' infinite times

```
1   package BICT_DAY;
2   public class Main
3   {
4       static void p()
5       {
6           System.out.println("hello");
7           p();
8       }
9       public static void main(String args[])
10      {
11          p();
12      }
13  }
```

Example: A Java program to calculate the factorial of 5

```java
package BICT_DAY;
public class Main
{
    static int factorial(int n)
    {
        if(n==1)
            return 1;
        else
            return (n*factorial(n-1));

    }
    public static void main(String args[])
    {
        System.out.println("Factorial of 5 is"+factorial(5));
    }
}
```

Java Classes and Objects

What is a class in Java?

A Class is an object constructor or a "blueprint" for creating objects.OrClass is a group of variables and methods of different data types.

What is an object?

An object is any real-world entity that can be converted into a computer program. The object can be physical or logical (tangible and intangible). An example of an intangible object is the banking system. Examples of tangible objects include a chair, a bike, a marker, a pen, a table, and a car.

An object has three characteristics:

State: These are properties of an object. For example, a car's object state might include its color, current speed, number of wheels, etc. In programming, object states are represented by variables.
Behaviour: These are all actions that an object can perform. For example, for a car, object behaviour will include slowing down, speeding up, and turning around. In programming, an object's behaviour is represented by functions.
Identity: It is a unique name for an object that differentiates an object from the rest

Declaring an object

Classname objectname=new Classname ();

Example of a Class Student

```
Class Student
{
        //Instance variables
        Static/Non static Variable 1;
        Static/Non static Variable 2;

        Static/Non function 1(parameter1, parameter2…..parametern)
        {
          //Local variables
           Variable 1;
           Variable 2
        }
        Static/Non Function 2(parameter1, parameter2…..parametern)
        {
          //Local variables
           Variable 1;
           Variable 2

        }
        Public static void main (String args[])
        {
            //The declarations of object "c"
        Student  c=new Student();

        }
}
```

Note the following:

1) Declaring Objects is also called instantiating a class
2) **An instance variable**: A variable which is created inside the class but outside the method and not declared as **static**. Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.
   - **Characteristics of instance variables**
     i) Instance variable can be accessed directly by any non-static method. Static method cannot access instance variable accept the main method
     ii) The main method can access the instance variables using an object
3) **Static variable:** Avariable which is created inside the class but outside the method and declared as **static.**
   - **Characteristics of static variables**
     i) Static variable can be accessed directly by any method regardless it is a static or not without an object
4) **Local variable:** Avariables which created within or inside a function block or function parameters.
   - **Characteristics of local variables**
     i) These variables can only be accessed within the function in which they are declared. The lifetime of the local variable is within its function only, which means the variable exists till the function executes.
5) **Instance methods/functions:** It is a method that is declared in the class without using the static keyword.
   - **Characteristics of Instance Method**
     i) Instance method can be accessed by static functions using object.
     ii) Instance method can be accessed by other instance methods without using object.
6) **Static methods/functions:** It is a method that is declared in the class using the static keyword.

- **Characteristics of Static Method**
  - i) It can access other static Methods and Variables without creating an object.
  - ii) It can not access an Instance Method or Variable directly. However, it can access an instance method or variable through an object.

Example: A Java program to print student name and subjects scores using object concept

```java
package BICT_DAY;

public class Main
{
    static String name="Onesno";
    public void modules()
    {
        int mathematics=90;
        int geography=40;
        System.out.println("Student Name is "+name);
        System.out.println("Mathematics Marks is "+mathematics);
        System.out.println("Geograph Marks  is"+geography);
    }


    public static void main(String args[])
    {
        Main xc=new Main();
        System.out.println("Student Name is "+name);
        System.out.println("..........Student Results are......... ");
        xc.modules();
    }
}
```

Example: A Java program to print student name and subjects scores using object and passing of variables concept

```java
package BICT_DAY;

public class Main
{
    String name="Onesno";
    public void modules(int math,int geo)
    {
        int mathematics=math;
        int geography=geo;
        System.out.println("Student Name is "+name);
        System.out.println("Mathematics Marks is "+mathematics);
        System.out.println("Geograph Marks  is"+geography);
    }


    public static void main(String args[])
    {
        Main xc=new Main();
        System.out.println("Student Name is "+xc.name);
        System.out.println("...........Student Results are..........  ");
        xc.modules(90,40);
    }
}
```

In real-time development, we create classes and use it from another class. It is a better approach than previous one. Let's see a simple example, where we are having main () method in another class. We can have multiple classes in different Java files or single Java file. If you define multiple classes in a single Java source file, it is a good idea to save the file name with the class name that has the main () method.

Example: A program implementing the main function in another class

```
1   package BICT_DAY;
2
3   class studentResults{
4       String name="Onesno";
5       public void modules()
6       {
7           int mathematics=90;
8           int geography=40;
9           System.out.println("Student Name is "+name);
10          System.out.println("Mathematics Marks is "+mathematics);
11          System.out.println("Geograph Marks  is"+geography);
12      }
13
14  }
15
16  public class Main
17  {
18      public static void main(String args[])
19      {
20          studentResults xc=new studentResults();
21
22          System.out.println("..........Student Results are..........");
23          xc.modules();
24      }
25  }
```

Constructors

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Example: A program to implement a constructor without passing parameters

```
1   package BICT_DAY;
2
3   public class Main
4   {
5       String name="Onesno";
6       Main()
7       {
8           int mathematics=90;
9           int geography=40;
10          System.out.println("Student Name is "+name);
11          System.out.println("Mathematics Marks is "+mathematics);
12          System.out.println("Geograph Marks  is"+geography);
13      }
14
15
16      public static void main(String args[])
17      {
18          System.out.println("...........Student Results are..........  ");
19          Main xc=new Main();
20      }
21  }
```

Example: A program to implement a constructor with the passing of parameters

```
1   package BICT_DAY;
2
3   public class Main
4   {
5       String name="Onesno";
6       Main(int math, int geo)
7       {
8           int mathematics=math;
9           int geography=geo;
10          System.out.println("Student Name is "+name);
11          System.out.println("Mathematics Marks is "+mathematics);
12          System.out.println("Geograph Marks  is"+geography);
13      }
14
15
16      public static void main(String args[])
17      {
18          System.out.println("..........Student Results are......... ");
19          Main xc=new Main(90,40);
20      }
21  }
```

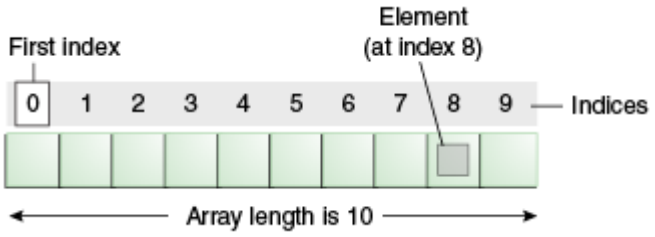
Array Data Structure

Java array is an object containing elements of a similar data type. Additionally, the elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. Depending on its size, we can store only a fixed set of elements in a Java Array. There are two types of arrays, namely.:

i Single-dimensional Array
ii Multi-dimensional Array

Single Dimensional Array

Single-dimension Array stores Elements in linear order (rows only). It stores the 1st Element at the 0 index, the 2nd at the 1 index, and so on.
Consider a description of an array named data[10] below;



There are three (3) syntaxes for declaring a single-dimension array
i Data_type arr[ ]; or
ii Data_type [ ]arr; or
iii Data_type [ ] arr

The **data_type** can be primitive data types like **int, char, double, byte, etc., or Java objects.** The **arr** is an identifier/Array name. An array name can be any name that follows the rules for naming variables.

But how many elements can this Array hold?

That is an excellent question! In Java, we must allocate memory for the Array to define the number of elements it can hold. The process of allocating the memory size of a Java Array is also known as **instantiation**. For example, an example,

arr=new Data_type [Array size];

For example, Separately **declaring** and **instantiates** an Array named "data" as an integer of size 10 using the first syntax

```
int data[ ];            //Array declaration
data=new int[10];  //Array initialization
```

Also, Arrays can be declared and initialized at the same time.

i Data_type arr[ ]=new Data_type [Array size] ;

ii Data_type [ ]arr=new Data_type [Array size] ;

iii Datatype[ ] arr=new Data_type [Array size] ;

For example, parallel **declares and instantiates** an Array named "data" as an integer of size 10 using all syntaxes.

```
i    int data[ ]=new int [10] ;
ii   int  [ ]data =new int[10] ;
iii  int[ ]  data =new int [10] ;
```

How do you initialize Single-dimensional arrays in Java?

Java Array initialization is also known as storing Elements in Java Array. Consider storing the Elements 100, 234,100,4,19,100,345 as integers to an Array named as **data.** Various mechanisms can be used as follows;

First Mechanism

```
int data[ ];            // Declaration
data=new int[7];   // instantiation
data[0]=100;        // Initializations
data[1]=234;
data[2]=100;
data[3]=4;
data[4]=19;
data[5]=100;
data[6]=345;
```

Second Mechanism

```
int data[ ]=new int[7];   // Declaration and instantiation.
data[0]=100;               // Initializations
data[1]=234;
data[2]=100;
data[3]=4;
data[4]=19;
data[5]=100;
data[6]=345;
```

Third Mechanism

```
int data[ ]=new int[ ]{ 100, 234,100,4,19,100,345 }     // Declaration, instantiation and Initialization
```

Fourth Mechanism

int data[ ]={ 100, 234,100,4,19,100,345 } // Declaration, instantiation and Initialization

Question: Write the program to perform the following to an Array named "data" below

An array "int data[]=new int[7]"

Elements	100	234	100	4	19	100	345
Index	0	1	2	3	4	5	6

i Store the given Elements in an array "data".

ii Display the 3rd element.

iii Display the Elements at index 2.

iv Update the Element 234 to be 5000

v Display all Elements Using a for-loop statement

Program: Storing and displaying Array Elements in Single-dimension Array

```
package AllPrograms;
import java.util.Scanner;
public class AllProgramsClass {

  public static void main(String[] args)
   {
            int data[]=new int[] {100, 234,100,4,19,100,345};
            //Displaying the third element
            System.out.println("The third Element is "+data[2]);

```

```
11                  //Displaying the Element at Index 2
12                  System.out.println("The Element at Index 2 is "+data[2]);
13
14                  //Updating the Element 234 to 5000
15                  data[1]=5000;
16
17                  //Displaying all Elements Using for-loop statement
18                  System.out.println("The following are the Elements:");
19                  for(int i=0;i<=6;i++)
20                  {
21                          System.out.println(data[i]);
22                  }
23          }
24      }
25
```

Output

```
The third element is 100
The Element at Index 2 is 100
The following are the Elements:
100
5000
100
4
19
100
345
```

Bubble Sort Algorithm Using an Array

Bubble sort is a simple sorting algorithm. This sorting algorithm is a comparison-based algorithm in which each pair of adjacent elements is compared, and the elements are swapped if they are not in order.

How the Bubble Sort Works?

An example on how to sort an array **int A[]=new int{14,33,27,35,10}** using bubble sort

1st Iteration

14, 33, 27, 35, 10,	Compare 14 and 33	14 <33	No Swap
14, 33, 27, 35, 10,	Compare 33 and 27	33 >27	Swap
14, 27, 33, 35, 10,	Compare 33 and 35	33 <35	No Swap
14, 27, 33, 35, 10,	Compare 35 and 10	35 >10	Swap

2nd Iteration

14, 27, 33, 10, 35,	Compare 14 and 27	14<27	No Swap
14, 27, 33, 10, 35,	Compare 27 and 33	27<33	No Swap
14, 27, 33, 10, 35,	Compare 33 and 10	33>10	Swap
14, 27, 10, 33, 35,	Compare 33 and 35	33<35	No Swap

3rd Iteration

14, 27, 10, 33, 35,	Compare 14 and 27	14<27	No Swap
14, 27, 10, 33, 35,	Compare 27 and 10	27>10	Swap
14, 10, 27, 33, 35,	Compare 27 and 33	27<33	No Swap
14, 10, 27, 33, 35,	Compare 33 and 35	33<35	No Swap

4th Iteration

14, 10, 27, 33, 35,	Compare 14 and 10	14>10	Swap
10, 14, 27, 33, 35,	Compare 10 and 14	10<14	No Swap

Now we should look into some practical aspects of bubble sort.

Bubble Sort Algorithm using an array

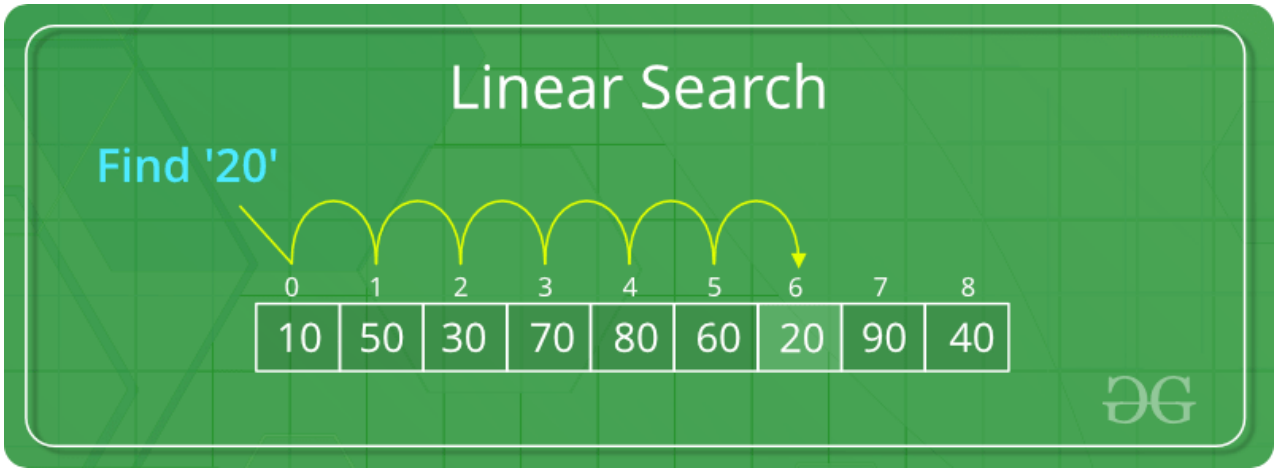
1. Declare an Array 'A.'
2. Store elements in the Array
3. Declare variable K, which stores intermediate/temporary results when swapping array contents
4. Sequentially compare each element with the preceding one, e.g. A[n]>A[n-1]
5. If the A[n] is greater than A[n-1] swap A[n-1] to A[n] and A[n-1] to A[n] with the help of variable K. Otherwise do nothing
6. Repeat steps 3 to 5 until all elements in the array are sorted

A Program to implement bubble sort

```
1   package BICT_DAY;
2   public class Main
3   {
4       public static void main(String args[])
5       {
6           int A[]=new int[]{16,10,4,6,2};
7           for(int i=0;i<=4;i++)
8           {
9               for(int j=0;j<=3;j++)
10              {
11                  int K;
12                  K=A[j];
13                  if(A[j]>A[j+1])
14                  {
15                      A[j]=A[j+1];
16                      A[j+1]=K;
17                  }
18              }
19          }
20          System.out.println("Sorted Array is");
21          for(int k=0;k<=4;k++)
22          {
23              System.out.println(A[k]);
24          }
25      }
26  }
```

Sequential Search Algorithm Using an Array

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found. The search continues till the end of the data set. It is the easiest searching algorithm.



Sequential search algorithm

1. Declare an Array 'A'
2. Store elements in the Array 'A'
3. Declare variable K and store the number to be searched
4. Sequentially compare each element in the array with variable K, e.g., A[n]= =K, A[n+1] = =K,
   A[n+2] = =K until the element found
5. Return 1 if the element is found; otherwise, return -1
6. Display to the user that the element was found

Example: a Java Program to search for the number 20 in a group of numbers

```java
package BICT_DAY;
public class Main
{
    static int findvalue(int A[],int K)
    {
        for(int i=0;i< A.length;i++)
        {
            if(A[i]==K)
                return 1;
        }
        return -1;
    }
    public static void main(String args[])
    {
        int A[]=new int[]{10, 50, 30, 70, 80,60,20,90,40};
        int K=20;
        if(findvalue(A,K)==1)
        {
            System.out.println("The element Found");
        }
        else
        {
            System.out.println("The element not Found");
        }
    }
}
```

Radix Sort Algorithm

Radix sort is a linear sorting algorithm that is used for integers. In Radix sort, digit-by-digit sorting is performed from the least significant to the most significant digit. Radix sort is a sorting algorithm that sorts elements by first grouping the digits at the same **place value**. Then, sort the elements in increasing or decreasing order.

Suppose we have an array of 10 elements. First, we will sort the elements by the value of the unit place. Then we will sort the elements by the value in the tenth place. This process goes on until the last significant place.

The Radix sort algorithm

1. Define 10 queues, each representing a bucket for each digit from 0 to 9.
2. Consider the least significant digit of each number in the list that is to be sorted.
3. Insert each number into its respective queue based on the least significant digit.
4. Group all the numbers from queue 0 to queue 9 in the order they were inserted into their respective queues.
5. Repeat from step 3 based on the next least significant digit.
6. Repeat from step 2 until all the numbers are grouped based on the most significant digit.

Example: Sorting the following list: 132, 543, 783, 063, 007, 898, 049

Step 1: Considerthe numbers in one space

13**2**, 54**3**, 78**3**, 06**3**, 00**7**, 89**8**, 04**9**

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
		13**2**	54**3**				00**7**	89**8**	04**9**
			78**3**						
			06**3**						

Step 2: Considerthe numbers at the tens space

1**3**2, 5**4**3, 7**8**3, 0**6**3, 0**0**7, 8**9**8, 0**4**9

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0**0**7			1**3**2	5**4**3		0**6**3		7**8**3	8**9**8
				0**4**9					

Step 2: Considerthe numbers at the hundreds space

007, **1**32, **5**43,**0**49, **0**63, **7**83,**8**98

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
007	**1**32				**5**43		**7**83	**8**98	
049									
063									

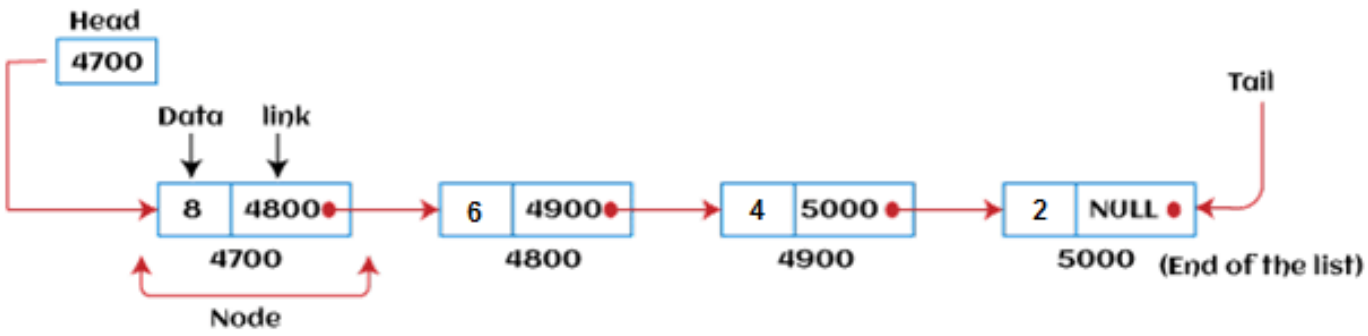
The Sorted array is 007,049,063,132,543,783,898

Linked list data structure

A linked list is a linear data structure that includes a series of connected nodes. A linked list is a set of nodes stored in random locations in memory. A node in a linked list consists of two parts: the data part and the address part. The last node of the list points to null. After arrays, linked lists are the second-most-used data structure. In a linked list, each link points to another link.

Representation of a linked list

A linked list can be represented as a chain of nodes, where each node points to the next node in the list. The representation of the linked list is shown below -



So far, we have been using an array data structure to organize a group of elements that are to be stored individually in memory. However, Array has several advantages and disadvantages that must be considered when deciding which data structure to use throughout the program.

Why use a linked list over an array?

A linked list is a data structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays -

- The size of the array must be known in advance before using it in the program.

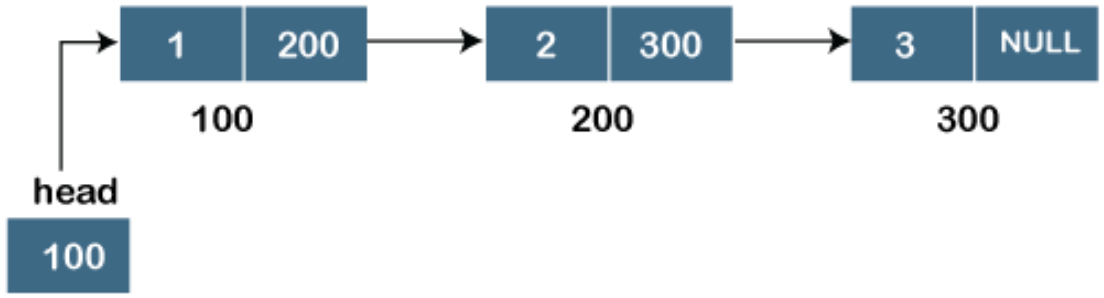
- Increasing the size of the array is a time taking process. It is almost impossible to expand the array at runtime.
- All the elements in the array need to be contiguously stored in the memory. Inserting an element into an array requires shifting all its predecessors.

A linked list is useful because -

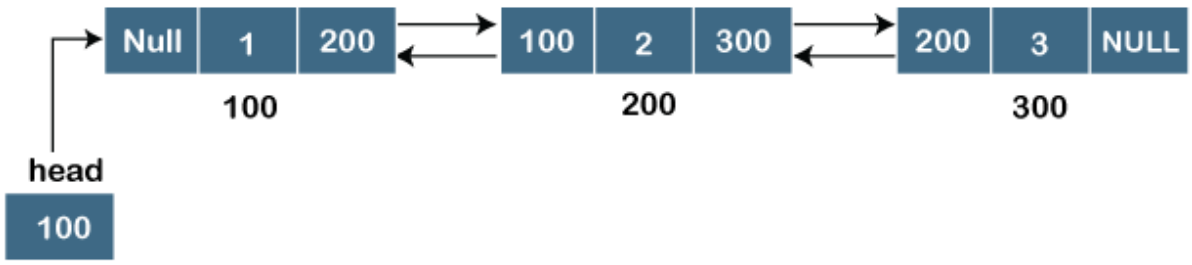
- It allocates the memory dynamically. All the nodes of the linked list are non-contiguously stored in memory and linked together using pointers.
- In a linked list, size is no longer a problem since we do not need to define its size at the time of declaration. The list grows as per the program's demand and is limited by available memory.

Types of Linked List

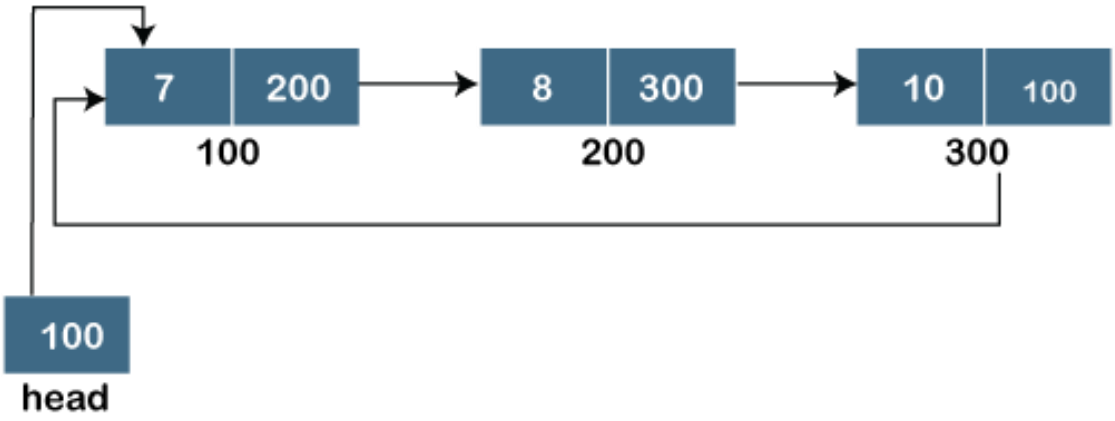
- **Singly-linked list -** A singly-linked list can be defined as a collection of an ordered set of elements. A node in the singly linked list consists of two parts: the data part and the link part. The data part of the node stores the actual information to be represented by the node, while the link part stores the address of its immediate successor.



- **Doubly linked list -** A doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).

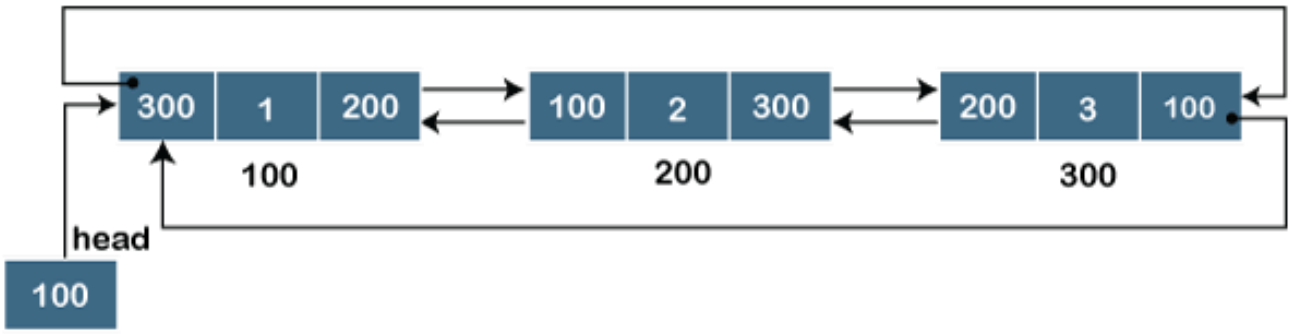


- **Circular singly linked list -** In a circular singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.



- **Circular doubly linked list -** A circular doubly linked list is a more complex type of data structure in which a node contains pointers to its previous node as well as

the next node. A circular doubly linked list doesn't contain NULL in any of the nodes. The last node of the list contains the address of the first node. The first node of the list also contains the address of the last node in its previous pointer.



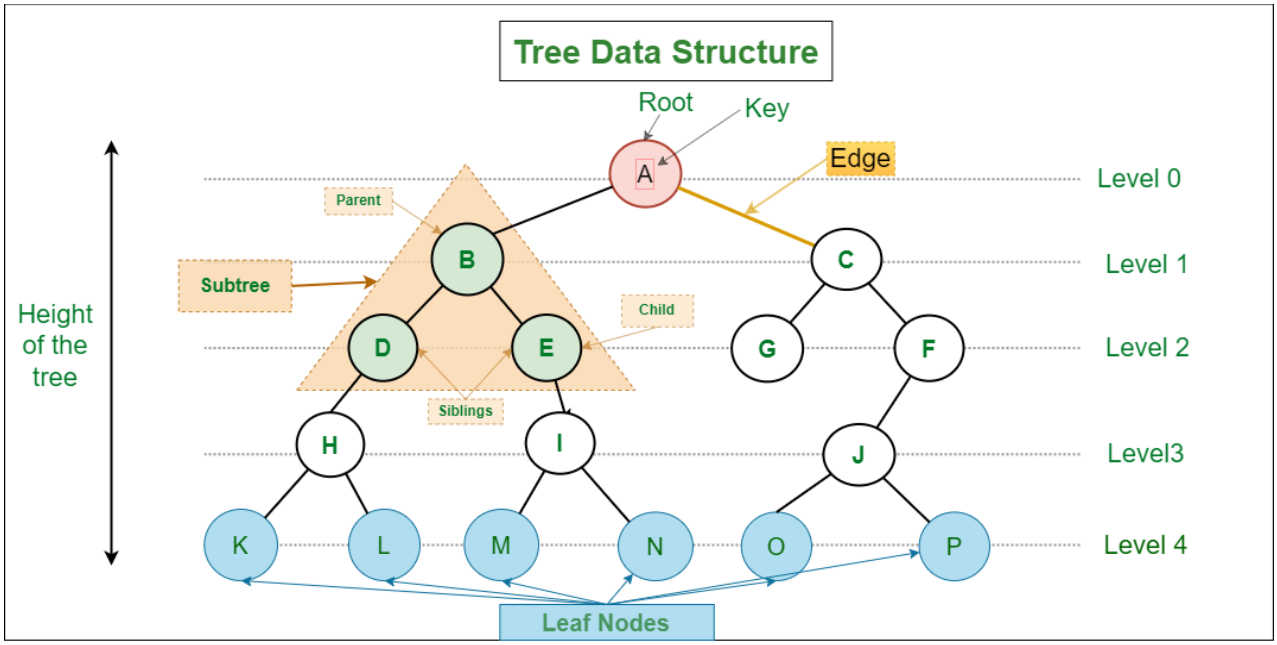
Operations performed on the Linked list

The basic operations that are supported by a list are mentioned as follows -

1. **Insertion** - This operation is performed to add an element to the list.
2. **Deletion -** It is performed to delete an operation from the list.
3. **Display** - It is performed to display the elements of the list.
4. **Search** - It is performed to search for an element from the list using the given key.

Tree Data Structure

A tree data structure is non-linear because it does not store data in a sequential order. In a tree data structure, data (Elements) are stored in the nodes organized in a hierarchy. It is a hierarchical structure, with elements arranged at multiple levels. In the tree data structure, you cannot navigate to the child node without traversing the parent node



Basic Terminologies In Tree Data Structure:

i. **Parent Node:** The node that is a predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.

ii. **Child Node:** The node that is the immediate successor of a node is called the child node of that node. Examples: **{D, E}** are the child nodes of **{B}.**

iii. **Root Node:** The topmost node of a tree, or the node that does not have any parent node, is called the root node. {A**}** is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.

iv. **Leaf Node or External Node:** The nodes that do not have any child nodes are called leaf nodes. **{K, L, M, N, O, P}** are the leaf nodes of the tree.

v. **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. **{A,B}** are the ancestor nodes of the node **{E}**

vi. **Descendant:** Any successor node on the path from the node to that leaf. **{E,I,M,N}** are the descendants of the node **{B}.**

vii. **Siblings:** Children of the same parent node are called siblings. **{D,E}** are called siblings.

viii. **Internal node:** A node with at least one child is called an internal node.

ix. **Subtree**: Any node of the tree along with its descendants.

Properties of a Tree:

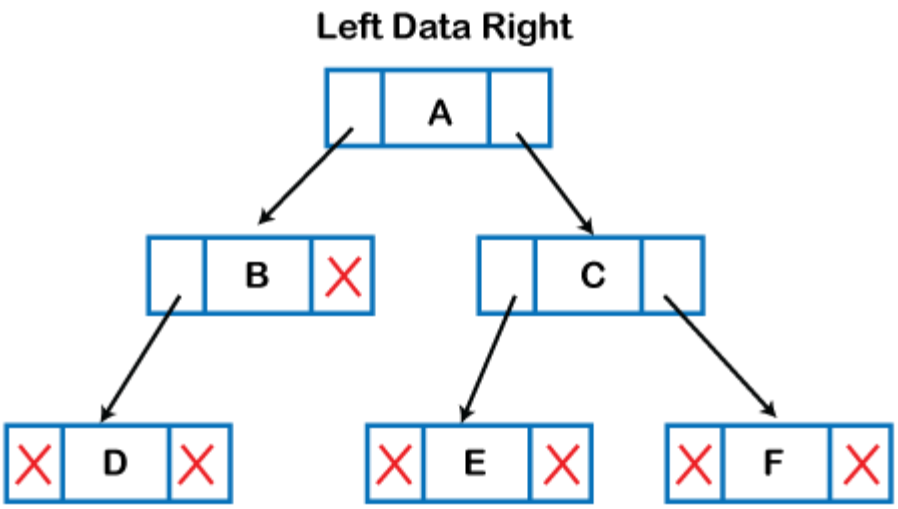
i. **Number of edges:** An edge can be defined as the connection between two nodes. If a tree has N nodes, then it will have (N-1) edges. There is only one path from each node to any other node of the tree.

ii. **Depth of a node:** The depth of a node is defined as the length of the path from the node to the root node. Each edge adds 1 unit of length to the path. So, it can also be defined as the. Therefore, the depth of root nodes is 0. E.g., Depth of Node **H** is **3**

iii. **Height of a node:** The height of a node can be defined as the length of the longest path from the node to the leaf of the tree. E.g. Height of **H** is**1,** height of **K** is **0, height of A(**height of a tree**)** is **4**

Implementation of Tree

The tree data structure can be implemented by dynamically creating nodes using pointers. The tree in memory can be represented as shown below:



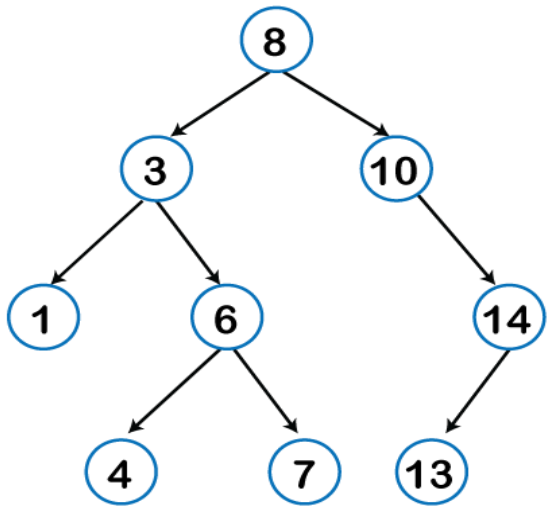
The figure above shows the representation of the tree data structure in memory. In the above structure, the node contains three fields. The second field stores the data; the first field stores the address of the left child, and the third field stores the address of the right child.

Applications of trees

i. Storing naturally hierarchical data: Trees are used to store the data in a hierarchical structure. For example, the file system. The file system on the disc drive stores files and folders as naturally hierarchical data in a tree structure.
ii. Routing table: Tree data structures can be used to represent the topology of a network and to calculate routing tables for efficient data transmission.
iii. For information indexing in a relational database.
iv. In websites to create XML and HTML documents
v. A binary search tree is a tree that allows fast search, insert, and delete on sorted data. It also allows finding the closest item
vi. Spanning Trees and shortest path trees are used in routers and bridges, respectively, in computer networks

Binary tree

Binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



Properties of a Binary Tree

- At each level of i, the maximum number of nodes is 2^i.
- The height of the tree is defined as the longest path from the root node to the leaf node. The tree which is shown above has a height equal to 3. Therefore, the maximum number of nodes at height 3 is equal to (1+2+4+8) = 15. In general, the maximum number of nodes possible at height h is (2^0 + 2^1 + 2^2+….2^h) = **Tpnh**=2^{h+1} -1
- Also, number of Levels (L)=h+1. Therefore **Tpnh**=2^L -1
- If the number of nodes is minimum, then the height of the tree would be maximum. Conversely, if the number of nodes is maximum, then the height of the tree would be minimum.

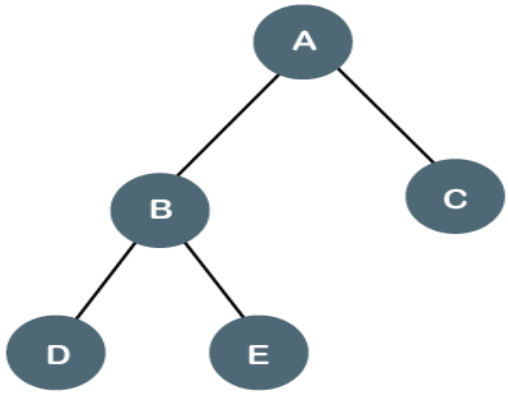
Types of Binary Tree

There are four types of Binary tree:

1. **Full/ proper/ strict Binary tree**

The full binary tree is also known as a strict binary tree. The tree can only be considered as the full binary tree if each node must contain either 0 or 2 children. The full binary tree can also be defined as the tree in which each node must contain 2 children except the leaf nodes.

Example:

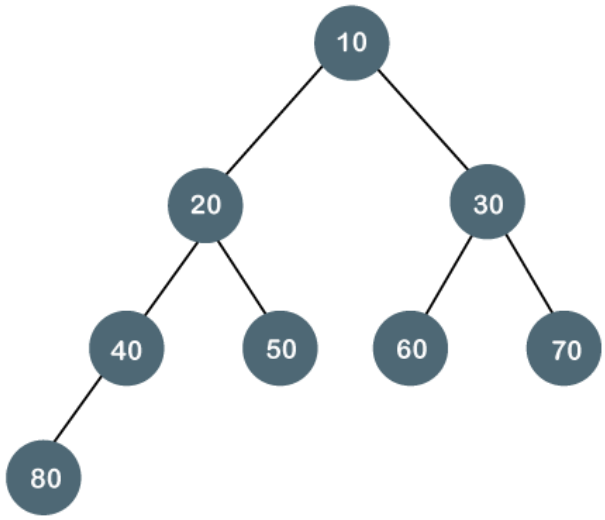


2. **Complete Binary Tree**

The complete binary tree is a tree in which all the nodes are completely filled except the last level. In the last level, all the nodes must be as left as possible. In a complete binary tree, the nodes should be added from the left.

The above tree is a complete binary tree because all the nodes are completely filled, and all the nodes in the last level are added at the left first.

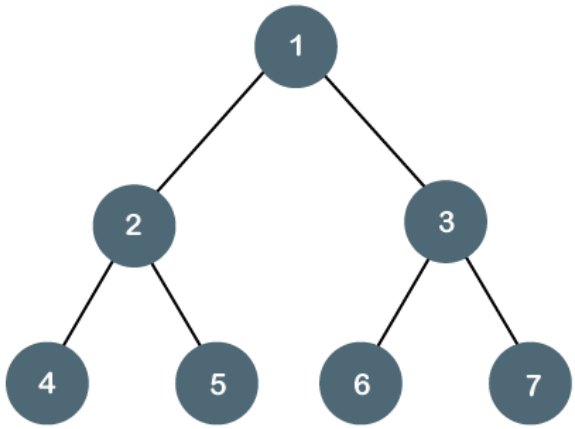
Example:



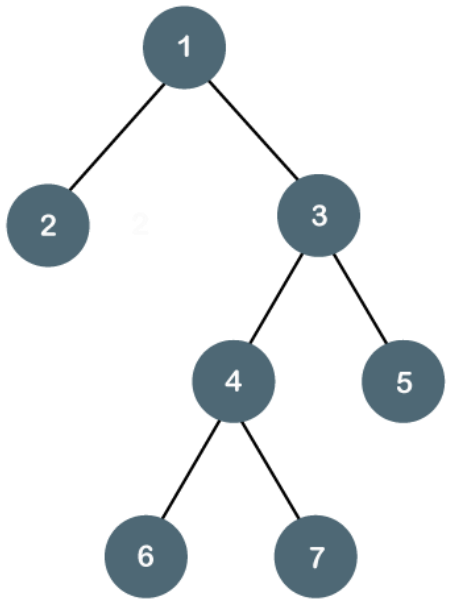
3. Perfect Binary Tree

A tree is a perfect binary tree if all the internal nodes have 2 children, and all the leaf nodes are at the same level.

Example:



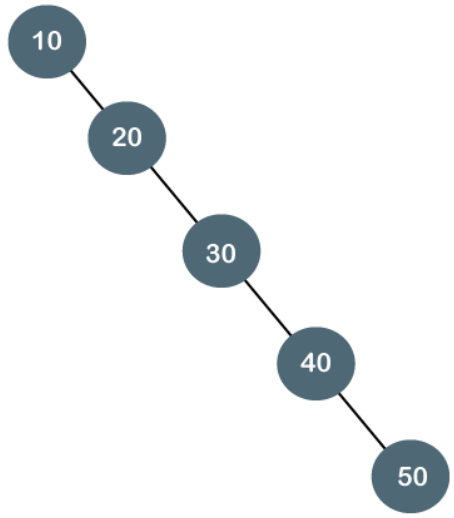
The below tree is not a perfect binary tree because all the leaf nodes are not at the same level.



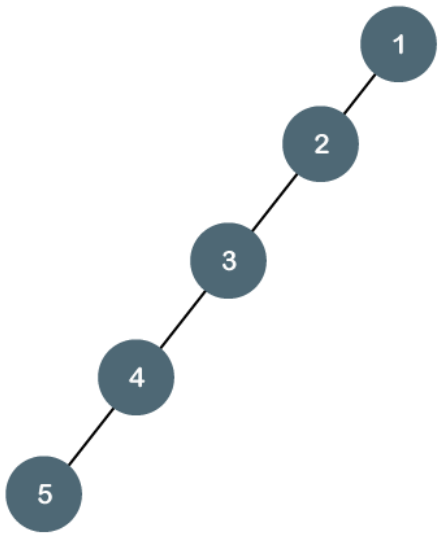
4. Degenerate Binary Tree

A degenerate binary tree is a tree in which all internal nodes have exactly one child.

Example.



The above tree is a degenerate binary tree because all the nodes have only one child. It is also known as a right-skewed tree as all the nodes have a right child only.

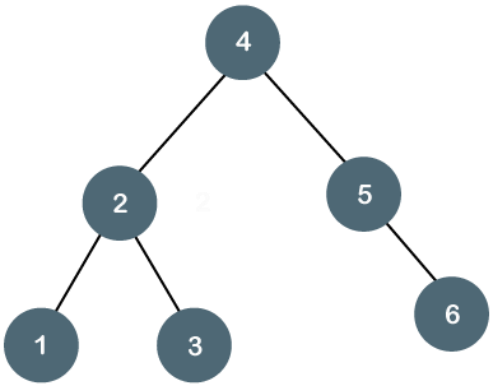


The tree above is also a degenerate binary tree because all nodes have only one child. It is also known as a left-skewed tree, as all nodes have only a left child.

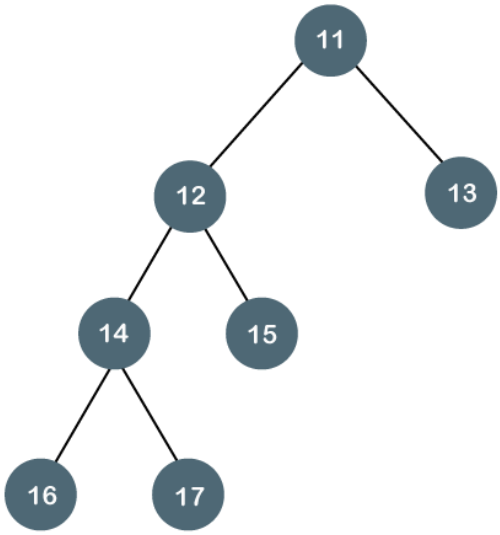
Balanced Binary Tree

The balanced binary tree is a tree in which both the left and right subtrees differ by at most 1. For example, ***AVL*** and ***Red-Black trees*** are balanced binary trees.

Examples



The above tree is a balanced binary tree because the difference between the left subtree and the right subtree is zero.



The above tree is not a balanced binary tree because the difference between the left and right subtrees is greater than 1.